

SPECIFICATION DRIVEN DOCUMENT

Arquitectura funcional y lógica operacional

RegistroHoras_Aloncar_V1

Sistema de extracción y normalización de reportes de horas mediante lenguaje natural

Cliente: Aloncar

Versión del documento: 2.0 (final)

Fecha: Junio 2026

Estado: MVP implementado y validado en producción

Tabla de contenidos

1. Objetivo del flujo
 2. Problema de negocio que resuelve
 3. Contexto operacional
 4. Alcance
 5. Entradas del sistema
 6. Salidas del sistema
 7. Reglas de negocio
 8. Flujo lógico operacional
 9. Estructura de datos
 10. Relaciones entre entidades
 11. Manejo de errores
 - 11 bis. Sistema de alertas (fase diferida)
 12. Auditoría y trazabilidad
 13. Dependencias críticas
 14. Riesgos operativos
 15. Escalabilidad futura
 16. KPIs sugeridos
 17. Recomendaciones arquitectónicas
 18. Lecciones de implementación (post-MVP)
- Anexo A — Glosario operativo
- Anexo B — Ejemplos de mensajes y output esperado
- Anexo C — Contrato de datos CargarCatalogos
- Anexo D — Catálogo de contactos por rol (preparado para V2.1)
- Anexo E — Mapa de nodos del flujo n8n implementado

1. Objetivo del flujo

El sistema RegistroHoras_Aloncar_V1 transforma reportes de trabajo redactados en lenguaje natural por los encargados del astillero en registros estructurados, normalizados y auditables dentro del sistema de control de horas del astillero Aloncar.

El sistema actúa como una capa de comprensión semántica entre el lenguaje informal del taller (mensajes de WhatsApp transcritos o tipeados en Slack, con errores ortográficos, abreviaturas, formatos heterogéneos y datos implícitos) y la estructura tabular rígida que requiere la planilla de control de horas para fines contables, estadísticos y de facturación.

1.1 Objetivo primario

Eliminar la carga manual de transcripción de reportes de horas que realiza un encargado de carga de datos, reduciendo el tiempo operativo de procesamiento, los errores de transcripción, y centralizando la información en un sistema único de registro.

1.2 Objetivos secundarios

- Establecer una arquitectura desacoplada que permita migrar el almacenamiento de catálogos desde Google Sheets hacia Supabase sin reescribir la lógica de negocio.
- Establecer un mecanismo de auditoría granular que permita reconstruir cómo se procesó cada mensaje, qué decisiones tomó el agente de IA, y qué datos quedaron sin resolver.
- Permitir que el canal de entrada (hoy Slack, mañana WhatsApp vía Evolution API o webhook genérico) sea intercambiable sin afectar el resto del flujo.
- Reducir la fricción para los encargados: deben poder reportar como ya lo hacen hoy, sin aprender un formato nuevo.

2. Problema de negocio que resuelve

2.1 Situación actual

En el astillero Aloncar, los supervisores de personal reportan diariamente las horas trabajadas por sus equipos a través de mensajes informales (WhatsApp, llamadas, papel). Estos reportes llegan a un encargado de registrar, quien debe interpretar el contenido, identificar barcos, órdenes de trabajo, operarios, rubros y horas, y volcar esa información manualmente a una planilla de control.

2.2 Dolores identificados

Dolor	Impacto
Cuello de botella humano	Una sola persona concentra el conocimiento de interpretación. Si falta, la carga se acumula.
Riesgo de error de transcripción	Errores manuales en horas, operario u OT impactan directamente sobre facturación y costos.
Pérdida de trazabilidad	El mensaje informal se pierde una vez transcripto. No queda registro auditable del input crudo.

Dolor	Impacto
Inconsistencia de nomenclatura	Errores ortográficos y abreviaturas no normalizadas se propagan a la planilla.
Velocidad de carga limitada	El tiempo de procesamiento por mensaje es alto y no escala con el volumen del astillero.
Conocimiento tácito no formalizado	Las reglas de interpretación viven en la cabeza del encargado.

2.3 Hipótesis de solución

Un agente de IA con catálogos del dominio en contexto puede replicar el 80-90% del trabajo interpretativo del encargado, dejando como excepciones flageadas únicamente los casos verdaderamente ambiguos. La capa de validación cruzada determinística sobre los catálogos garantiza que ningún dato escrito en planilla esté fuera del conjunto canónico de entidades del negocio.

El sistema no reemplaza al encargado de control: lo libera del trabajo repetitivo de transcripción y lo posiciona como revisor de excepciones, donde su criterio aporta más valor.

3. Contexto operacional

3.1 Dominio del negocio

Aloncar es un astillero que ejecuta reparaciones y mantenimientos sobre embarcaciones (barcos), organizando el trabajo en órdenes de trabajo (OT). Sobre cada OT se asignan operarios propios del astillero (grupo ASTILLERO en el catálogo de personal) o contratistas externos (identificados con un grupo de trabajo propio: TOBARES, RAMON ARANDA, AVALOS, etc.).

El trabajo se ejecuta por rubros (raschinaje, calderería, carpintería, mecánica, pintura, pala, grúa, etc.). Cada operario tiene un rubro principal en el catálogo, pero puede ejecutar tareas de otros rubros según la necesidad operativa.

3.2 Actores del sistema

El reporte de horas atraviesa una cadena de actores antes de llegar al sistema. Solo uno de ellos queda registrado como emisor en el log.

Actor	Rol en el sistema
Supervisor de personal de planta	Genera el reporte primario de horas de su equipo y lo envía por WhatsApp al encargado de registrar. No interactúa con el sistema.
Encargado de registrar (emisor Slack)	Recibe los reportes por WhatsApp, los lee, hace ajustes mínimos y los publica en el canal Slack. Es el único actor que el sistema captura como usuario_emisor en el log.
Sistema (RegistroHoras_Aloncar_V1)	Interpreta el mensaje, normaliza entidades, valida contra catálogos y escribe registros estructurados.
Encargado de control de horas	Revisa la planilla destino, atiende filas marcadas con R y corrige.
Supervisor	Aprueba la planilla finalizada para facturación y liquidación.
Administrador de catálogos	Mantiene actualizados los catálogos de personal, barcos, rubros y órdenes de

Actor	Rol en el sistema
	trabajo.

Trazabilidad del emisor: el sistema registra el ID del usuario Slack que publica el mensaje (el encargado de registrar), no al supervisor de planta que originó el reporte. El diseño soporta múltiples usuarios Slack desde V1. Cuando se migre a WhatsApp, este identificador pasará a ser el número de teléfono del emisor.

3.3 Volumen estimado y frecuencia

- Frecuencia de mensajes: diaria, con picos al final del día laboral.
- Volumen estimado: 5 a 30 mensajes por día.
- Catálogo de personal: aproximadamente 200 entradas (de las cuales se cargan solo las ACTIVO, ~160 en la práctica).
- Catálogo de barcos: decenas de embarcaciones activas en cualquier momento.
- Catálogo de rubros: cerrado, alrededor de 30 rubros estandarizados.
- Catálogo de OTs activas: filtrado dinámicamente, decenas a baja centena.

4. Alcance

4.1 Dentro de alcance (V1 — implementado)

- Recepción de mensajes desde un canal Slack específico (COB6K792PED) vía webhook con Event Subscriptions.
- Filtrado de mensajes del propio bot para evitar loops de auto-respuesta.
- Extracción de fecha del mensaje; rechazo bloqueante si no se encuentra fecha.
- Carga de catálogos de personal (solo ACTIVO), barcos, rubros y OTs activas desde Google Sheets.
- Extracción semántica de entidades por LLM (gpt-4o-mini) con catálogos en contexto.
- Validación cruzada determinística contra catálogos, incluyendo selección de OT por habilitación real (columna G).
- Cálculo de distribución de horas en normales / 50% / 100% según calendario y tipo de operario.
- Escritura quirúrgica en la planilla de horas detectando la última fila real (columna B), con marcador R para revisión.
- Escritura del mensaje_id (columna AD) y del motivo de revisión del sistema (columna AE), separado de las observaciones de obra (columna T).
- Registro paralelo en hoja de log con trazabilidad completa del procesamiento.
- Respuesta de confirmación al emisor en el mismo canal Slack.

4.2 Fuera de alcance (V1, candidatos a versiones posteriores)

- Validación de cumplimiento de target diario de horas por operario. Lógica diseñada, implementación diferida.
- Trigger alternativo por WhatsApp vía Evolution API (preparado, sin conectar).

- Sistema de alertas por rol a jefes de obra / supervisores (nodo E9b, diferido a V2.1, ver Sección 11 bis).
- Migración de catálogos, log y planilla destino a Supabase.
- Mecanismo de retry asíncrono para mensajes incompletos (sistema de pendientes con reanudación).
- Interfaz web para revisión y corrección de filas marcadas con R.
- Reglas de negocio por OT (rubros permitidos por tipo de orden), pendientes de mapeo en el proyecto de aplicación.

5. Entradas del sistema

5.1 Entrada principal: mensaje Slack

El sistema recibe un mensaje de texto plano publicado en el canal de Slack designado (C0B6K792PED) por el encargado de registrar. El payload de Slack, recibido vía webhook con Event Subscriptions, anida los datos del mensaje bajo body.event. Los atributos relevantes:

Atributo	Ubicación en payload	Uso
text	body.event.text	Procesamiento por LLM + persistencia en log.
user	body.event.user	Trazabilidad del emisor (encargado de registrar). Soporta múltiples usuarios.
ts	body.event.ts	Construcción del mensaje_id de auditoría.
channel	body.event.channel	Filtro de seguridad (debe ser C0B6K792PED).
bot_id	body.event.bot_id	Si está presente, el mensaje es del bot y se descarta (evita loop).

5.2 Entradas secundarias: catálogos de dominio

El sistema lee, en cada ejecución, los catálogos que definen el universo de entidades válidas, mediante el sub-workflow CargarCatalogos. CargarCatalogos lee de solo dos spreadsheets; la planilla de horas queda exclusivamente como destino de escritura. El detalle completo de fuentes, columnas y filtros está en el Anexo C.

Catálogo	Fuente actual	Contenido y filtro
Personal	B.D.NewSystemm → PersonalClasificacionCostos	Nombre, grupo, U\$/hora, rubro principal. Solo registros con ESTADO = ACTIVO.
Rubros	B.D.NewSystemm → BaseDatosP/PlanillasRegistro, col C	Lista cerrada. Fuente de verdad de la cual se exporta RUBROS_OtrosDesplegables_IMPORTADA.
Barcos	LISTA_TRABAJO → UNIFICADO, col A	Valores únicos no vacíos, filtrados por estado de OT = "En ejecución/En curso".
OTs activas	LISTA_TRABAJO → UNIFICADO	Tuplas (barco, OT, descripción, contratistas, comentario, jefe de obra), estado = "En ejecución/En curso".

Los rubros se leen de su fuente de origen (B.D.NewSystemm), no del export derivado. Esto elimina la dependencia de que el export se haya ejecutado. La lógica del flujo no asume nada sobre el medio de almacenamiento: el contrato es que CargarCatalogos devuelve siempre un objeto JSON con la misma forma.

6. Salidas del sistema

6.1 Salida primaria: filas en planilla de horas

Cada tarea identificada genera una fila en la hoja principal de HORAS_PLANILLAS_DE_REGISTRO. Regla de consolidación: una fila por combinación única de (fecha, operario, barco, OT, rubro). Múltiples menciones de la misma combinación se suman en horas y concatenan en observaciones.

Ejemplo de consolidación

Si un mensaje reporta "Franco — raschinaje — Caliz — achique sentina — 3h" por la mañana, y más abajo "Franco — raschinaje — Caliz — achique sentina — 2h a la tarde":

Sin consolidación: dos filas (3h y 2h) para la misma persona, barco, OT, rubro y fecha.

Con consolidación: una sola fila con 5h totales, observaciones concatenadas.

Col	Header	Contenido
A	APROBACIÓN	"R" si la fila requiere revisión; vacío en caso contrario.
B	FECHA	Fecha de ejecución reportada, formato DD/MM/YYYY. Ancla para detectar última fila real.
C	BARCO	Nombre canónico del barco (del catálogo).
D	ORDEN DE TRABAJO	OT canónica, seleccionada por habilitación real (no solo por texto).
E	CONTRATISTA	Nombre canónico del operario o contratista ejecutor.
F	TIPO TRAB	HORA (default) / PRESUPUESTO / CONTROL, inferido del comentario de la OT (RN-14).
G	NORMAL	Horas dentro del target diario.
H	H. AL 50%	Horas extras hasta 4h.
I	H. AL 100%	Horas extras por encima de 4h.
J	RUBRO	Rubro del trabajo (del mensaje, no del catálogo de personal).
K-R	(omitidas)	No se escriben.
S	N P.R.	Calculada por fórmula nativa. No se escribe.
T	OBSERVACIONES	Descripción de la tarea (del mensaje) con corrección ortográfica.
AD	mensaje_id	Identificador del mensaje originador. Trazabilidad bidireccional con el log.
AE	MOTIVO_REVISION_SISTEMA	Diagnóstico del sistema sobre por qué la fila quedó con R. Separado de las observaciones de obra.

Separación T vs AE: la columna T contiene solo lo que dijo el humano (observación de obra); la columna AE

contiene solo el razonamiento del sistema (por qué quedó marcada con R). Esta separación evita que el revisor tenga que leer texto técnico cuando solo quiere ver el comentario de obra.

6.2 Salida secundaria: fila en hoja de log

Cada mensaje procesado genera exactamente una fila en la hoja LOG_PROCESAMIENTO. Una entrada por ejecución, sin importar cuántas filas haya generado en la planilla principal.

Campo	Significado
mensaje_id	Identificador único = timestamp del mensaje + hash del texto crudo.
fecha_procesamiento	Cuándo se ejecutó el flujo.
fecha_reportada	Fecha extraída del mensaje.
usuario_emisor	ID del usuario Slack que publicó el mensaje (el encargado de registrar).
mensaje_literal	Texto crudo del mensaje, sin modificar.
salida_agente_texto	Versión legible de las tareas extraídas (barco OT contratista).
no_reconocido	Resumen de fragmentos no resueltos.
resumen_agente	Análisis breve del LLM sobre la completitud del mensaje.
total_filas	Cantidad de filas escritas en la planilla principal.
filas_review	Cantidad de filas marcadas con R.
llm_model	Modelo utilizado (gpt-4o-mini).
prompt_version	Versión del prompt (gestionada desde el nodo Set - PromptConfig).
tokens_in / tokens_out	Métricas de costo y volumen.

6.3 Salida terciaria: respuesta de confirmación

Un mensaje de respuesta en Slack confirma al emisor el resultado: cantidad de filas registradas, filas en revisión y datos no resueltos. Cierra el ciclo de feedback con el encargado.

7. Reglas de negocio

Las siguientes reglas viven en código determinístico o en validaciones cruzadas posteriores a la extracción semántica. No están sujetas a interpretación del LLM.

7.1 Reglas de horas

RN-01 — Target diario según calendario y tipo de operario

- Lunes a viernes, operario propio (grupo ASTILLERO): target = 9 horas.
- Lunes a viernes, contratista externo: target = 8 horas.
- Sábado y domingo: target = 0 (todo lo reportado es hora extra).

RN-02 — Distribución de horas extra

- Las primeras 4 horas extra se imputan a la columna H. AL 50%.
- A partir de la quinta hora extra inclusive, a la columna H. AL 100%.

```
horas_normal = min(horas_totales, target) horas_extra = max(0, horas_totales - target)
horas_50 = min(horas_extra, 4) horas_100 = max(0, horas_extra - 4)
```

Dependencia conocida: el cálculo de target depende de saber si el operario es ASTILLERO o externo. Si el operario no resuelve contra el catálogo, el sistema no puede determinar el target y por defecto asume 8h (externo). Esto produce un 8+1 en lugar de 9+0 para propios no reconocidos. La fila queda marcada con R, por lo que el revisor corrige; pero conviene tenerlo presente: horas mal distribuidas siempre van acompañadas de R por operario no resuelto.

RN-03 — Multiplicación de horas por operarios múltiples

Cuando un bloque menciona más de un operario para una misma tarea ("Luciano Enzo 2 horas limpieza"), las horas se imputan a cada operario individualmente (2h a Luciano y 2h a Enzo). Refleja que ambos trabajaron en paralelo.

7.2 Reglas de matching de entidades

RN-04 — Matching de operario

El operario reportado debe matchear contra un nombre canónico del catálogo. Orden de preferencia: (1) match exacto sin sensibilidad a mayúsculas/acentos; (2) match parcial sobre apellido o nombre; (3) desambiguación por rubro si hay homónimos; (4) si persiste la ambigüedad, el campo queda nulo y la fila se marca con R. Los homónimos sin desambiguar requieren revisión humana del mensaje original.

RN-05 — Matching de barco

El barco se identifica por mención literal. Si no aparece explícito, se deduce: (1) por arrastre de contexto del bloque; (2) por cruce de rubro del operario, OTs habilitadas y descripción de la tarea. Si la deducción produce varias opciones o ninguna, la fila se marca con R pero se carga la mayor cantidad posible de datos conocidos (operario, horas, descripción, tipo, rubro). Cargar parcialmente reduce el tiempo de corrección manual.

RN-06 — Matching de OT por habilitación (no solo por texto)

El 98% de los mensajes no menciona explícitamente la OT. La deducción procede en dos filtros: primero por similitud textual entre la tarea y la columna Descripción de las OTs del barco; luego, de esas candidatas, se conservan solo las que habilitan al operario o su grupo (ASTILLERO.{rubro}) en la columna G. Solo si queda exactamente una OT habilitada se resuelve; si quedan varias, es ambigüedad real (R); si ninguna habilita, la OT queda sin resolver (R), pero el resto de los datos se cargan.

Esta regla corrige un comportamiento erróneo de la versión inicial, donde el sistema tomaba la primera coincidencia textual sin verificar habilitación. Ejemplo real: un calderero reportando "parche sentina" matcheaba la OT SENTINA (que solo habilita raschinaje) en vez de CALDERERIA VARIOS (que sí lo habilita y también menciona sentina en su descripción). La validación de habilitación ahora elige la OT correcta o marca R, en lugar de escribir un dato incorrecto sin señalarlo.

RN-07 — Validación cruzada match_contratista

Una vez asignada una tupla (barco, OT, operario, rubro), el sistema verifica que el operario o su grupo (ASTILLERO.{rubro}) figure en la columna G de UNIFICADO para esa OT. Tras la corrección de RN-06, esta validación es una verificación de auditoría adicional, ya que la habilitación se aplica durante la selección de OT.

RN-08 — Rubro: origen y desambiguación

El rubro a escribir en la columna J es el rubro mencionado o inferido del mensaje, NUNCA el del catálogo de personal. El rubro del catálogo solo se usa como tie-breaker cuando hay ambigüedad de operario homónimo.

7.3 Reglas de procesamiento textual

RN-09 — Corrección ortográfica conservadora

El LLM corrige errores ortográficos evidentes en español al generar OBSERVACIONES ("limpiesa" → "limpieza"). No reinterpreta contenido: si una palabra es ambigua, preserva el original.

RN-10 — Trazabilidad del fragmento original

Cada tarea extraída lleva un campo raw_block con el fragmento literal del mensaje del cual fue derivada. Permite reconstruir auditorías y validar interpretaciones.

7.4 Reglas de control de flujo

RN-11 — Fecha bloqueante

Si el mensaje no contiene una fecha reconocible, el sistema no lo procesa. Responde al emisor solicitando que agregue la fecha y reenvíe el mensaje completo. No se guarda estado intermedio en V1.

RN-12 — Marcador R con escritura parcial

Toda fila que carezca de al menos un campo obligatorio (fecha, barco, OT, operario, rubro, horas) se marca con "R". El marcador no impide la escritura: la fila se escribe con todos los campos conocidos con certeza, y las celdas vacías solo donde no se pudo completar. El objetivo es minimizar el trabajo del revisor, que solo completa lo que falta.

RN-13 — Contenido no resuelto preservado

Cuando el LLM identifica fragmentos que no pudo asociar a ninguna fila, se genera una fila adicional con APROBACIÓN=R y el contenido literal en OBSERVACIONES. Ningún dato del mensaje se pierde silenciosamente.

RN-13b — Barco faltante a nivel fila

A diferencia de la fecha (bloqueante a nivel mensaje), el barco es atributo de cada fila. Si una tarea no tiene barco deducible, esa fila se marca con R y se carga con lo conocido, pero el resto de las filas del mensaje se procesan con normalidad. El barco es la única constante operativa del negocio; su ausencia es excepcional y se señala al encargado de registrar, sin detener el mensaje completo.

RN-14 — Inferencia del tipo de trabajo

El tipo de trabajo (columna F destino) se infiere del texto de la columna Comentario de UNIFICADO para la OT correspondiente:

- Comentario vacío o sin mención relevante → HORA (default).
- Comentario con referencia a presupuesto ("Presupuesto", "PRC", "presu") → PRESUPUESTO.
- Comentario con referencia a control de horas → CONTROL.

8. Flujo lógico operacional

El flujo se compone de etapas funcionales activas más una etapa diferida (E9b). Cada etapa tiene una responsabilidad única y comunica con la siguiente mediante un contrato de datos definido. Las etapas marcadas con [SWAP] son puntos de abstracción intercambiables; la marcada con [DIFERIDO] está especificada pero su implementación se posterga.

8.1 Diagrama lógico (implementado)

```
[Webhook] Recepción Slack Event Subscriptions [SWAP] | [E1] ExtractAndValidate
(lee body.event, filtra bot, extrae fecha) | [IF] tiene fecha? |-- NO --> [E1b]
Solicitar fecha al emisor --> FIN | -- SI --> [E2] CargarCatalogos (sub-workflow)
[SWAP] | [Set] PromptConfig (prompt_version + template editable) | [E3a]
PrepararPrompt --> [E3b] OpenAI --> [E3c] ParseLLMResponse | [E4] ValidacionCruzada
(matching + OT por habilitación) | [E5] CalcularHoras --> [E6] ConsolidarFilas |
[E6b] PrepararLecturaColB --> [GET] LeerColumnaB | [E7a] PrepararSheet (detecta
ultima fila real, arma rango) | [E7b] UpdatePlanilla (PUT por rango, no append)
| [E8] EscribirLog | | --> [E9] ResponderUsuario [SWAP] | --> [E9b]
GenerarYEnviarAlertas [DIFERIDO V2.1]
```

8.2 Descripción funcional por etapa

Webhook — Recepción Slack [SWAP]

Qué hace: Punto de entrada. Webhook genérico de n8n que recibe los eventos de Slack vía Event Subscriptions. Responde el handshake inicial (challenge) de Slack y, en operación normal, entrega el payload al resto del flujo. El payload anida el mensaje bajo body.event.

Por qué se diseñó así: Se usó un webhook genérico en lugar del nodo Slack Trigger por incompatibilidad de credencial en la versión de n8n (ver Sección 18). El nombre y la implementación del trigger son intercambiables.

E1 — ExtractAndValidate

Qué hace: Lee el texto desde body.event.text, filtra mensajes del propio bot (body.event.bot_id presente) para evitar loops, y extrae la fecha con regex (patrones DD/MM, DD-MM, DD de mes). Genera el mensaje_id.

Por qué se diseñó así: La validación de fecha es bloqueante por diseño: la fecha es el ancla temporal del régimen de horas y errores aquí se propagan a facturación. El filtro de bot es crítico: sin él, las respuestas del bot disparan nuevas ejecuciones en loop infinito.

E1b — Solicitar fecha al emisor

Qué hace: Si E1 no encuentra fecha, responde al emisor pidiendo que agregue la fecha y reenvíe. El flujo termina sin escribir.

Por qué se diseñó así: No se persiste estado de mensaje pendiente en V1. El costo de reenvío es bajo y simplifica la arquitectura.

E2 — CargarCatalogos [SWAP]

Qué hace: Sub-workflow que lee personal, rubros, barcos y OTs activas, y devuelve un objeto JSON estándar. Lee de dos spreadsheets (B.D.NewSystemm y LISTA_TRABAJOS).

Por qué se diseñó así: Punto crítico de la migración futura a Supabase. Mientras el contrato JSON de salida se mantenga, los nodos internos pueden reemplazarse sin tocar el flujo principal.

Set — PromptConfig

Qué hace: Nodo de configuración que define `prompt_version` y `prompt_template` como valores editables desde la UI, con placeholders (`{{PERSONAL}}`, `{{RUBROS}}`, `{{BARCOS}}`, `{{OTS}}`) que E3a reemplaza con los catálogos.

Por qué se diseñó así: Extrae el prompt del código JavaScript a un lugar visible y editable sin programar. Cada cambio al prompt incrementa `prompt_version`, lo que el log registra para A/B testing y rollback.

E3a/E3b/E3c — Extracción LLM

Qué hace: E3a arma el prompt inyectando los catálogos. E3b llama a la API de OpenAI (gpt-4o-mini, structured output JSON). E3c parsea la respuesta y la fusiona con los datos del mensaje.

Por qué se diseñó así: El structured output garantiza formato estable. Few-shot examples en el prompt enseñan a manejar los formatos heterogéneos de los encargados.

E4 — ValidacionCruzada

Qué hace: Valida cada tarea del LLM contra catálogos: existencia de operario, barco y rubro; selección de OT por habilitación (RN-06); inferencia de tipo_trab (RN-14); determinación de needs_review y motivo_revision.

Por qué se diseñó así: Convierte la inferencia probabilística del LLM en verdad verificable contra catálogos. Es la red de seguridad. La resolución de operario ocurre antes que la de OT, porque la OT depende de quién es el operario.

E5 — CalcularHoras

Qué hace: Aplica RN-01 y RN-02 en código determinístico: horas_normal, horas_50, horas_100 según día de la semana y tipo de operario.

Por qué se diseñó así: La aritmética nunca queda en manos del LLM, lo que evita errores de cálculo y permite auditar con seguridad.

E6 — ConsolidarFilas

Qué hace: Agrupa por (fecha, operario, barco, OT, rubro). Suma horas, concatena observaciones de obra y motivos de revisión en arrays separados, expande operarios múltiples.

Por qué se diseñó así: La consolidación es determinística, no depende del LLM. La separación de observaciones (T) y motivo de revisión (AE) ocurre aquí.

E6b / GET — Detección de última fila

Qué hace: E6b prepara la URL para leer solo la columna B de la hoja HORAS. El nodo GET hace la lectura. E7a recorre esos valores de atrás hacia adelante para encontrar la última fila con FECHA real.

Por qué se diseñó así: Resuelve el problema de que el método append de Sheets cuenta las fórmulas arrastradas (columna S) como datos, escribiendo en filas equivocadas. Anclar a la columna B (que solo se llena con un registro real) da control exacto.

E7a — PrepararSheet

Qué hace: Calcula el rango de destino (A{n+1}:AE{n+m}) y arma la matriz de 31 columnas, escribiendo solo A-J, T, AD, AE y dejando intactas K-S (incluida la fórmula de S).

Por qué se diseñó así: La escritura es quirúrgica: el flujo controla la fila exacta, no la heurística de Sheets.

E7b — UpdatePlanilla

Qué hace: Escribe las filas vía PUT a valores/{rango} con valueInputOption=USER_ENTERED, en lugar de append.

Por qué se diseñó así: El update por rango respeta la fila calculada y no salta a posiciones equivocadas. Requiere que la hoja tenga filas físicas disponibles (ver Sección 18).

E8 — EscribirLog

Qué hace: Genera una fila en LOG_PROCESAMIENTO con toda la trazabilidad del mensaje (operación append).

Por qué se diseñó así: La hoja de log convive con la planilla destino para facilitar auditorías cruzadas. Se revisará al migrar a Supabase.

E9 — ResponderUsuario [SWAP]

Qué hace: Envía confirmación al emisor con el resumen del procesamiento.

Por qué se diseñó así: El feedback inmediato cierra el ciclo y permite detectar problemas en caliente.

E9b — GenerarYEnviarAlertas [DIFERIDO V2.1]

Qué hace: Nodo placeholder. Especificado pero no implementado: leerá filas con R, las agrupará por destinatario según la matriz de roles (Sección 11 bis), resolverá el usuario Slack de cada rol y enviará un mensaje consolidado.

Por qué se diseñó así: Se difiere porque depende del catálogo de contactos por rol (Anexo D), que vivirá en Supabase. La estructura queda preparada para que su construcción sea directa.

9. Estructura de datos

9.1 Contrato de salida del sub-workflow CargarCatalogos

Define la forma de los catálogos que el resto del flujo consume. Las fuentes, columnas y filtros concretos están en el Anexo C.

```
{  "personal": [ { "nombre":"Hernández, Santiago", "grupo":"ASTILLERO",
"rubro":"MAQUINISTA", "usd_hora":9.97 } ],  "rubros":
[ "RASCHINAJE", "CALDERERIA", "PALA", "MECANICA", ... ],  "barcos":
[ "CALIZ", "LOBO", "ARTESANAL", ... ],  "ots_activas": [ { "barco":"CALIZ",
"ot":"TOBERA", "descripcion":"...",
"contratistas_col_g":"TOBARES, ASTILLERO.PALA",
"comentario":"...", "jefe_obra":"..." } ] }
```

9.2 Schema de salida del LLM

Schema JSON forzado vía structured output en la API de OpenAI. Cada tarea incluye raw_block, barco, ot_candidata, operario, rubro, horas, observacion, y campos de matching (match_barco, match_operario, match_rubro, match_ot, confidence). Además: campos_no_resueltos y resumen_agente a nivel mensaje.

9.3 Estructura enriquecida post-validación

Después de E4 y E5, cada item se enriquece con campos derivados:

Campo	Origen	Significado
barco_canonico / operario_canonico / ot_canonica / rubro_canonico	E4	Entidad canónica resuelta del catálogo, o null si no resolvió.
match_contratista	E4	valid_astillero valid_external mismatch not_found not_applicable
tipo_trab	E4	HORA PRESUPUESTO CONTROL (inferido del comentario de la OT, RN-14).
needs_review	E4	true si algún campo obligatorio falta o si hay inconsistencia.
motivo_revision	E4	Texto del diagnóstico del sistema (va a la columna AE).
deduced_fields	E4	Campos cuyo match no fue exacto.
horas_normal / horas_50 / horas_100	E5	Distribución de horas calculada.

10. Relaciones entre entidades

10.1 Diagrama lógico de entidades

```
BARCO *---* ORDEN_TRABAJO (relación N:M vía tabla puente) |
+-----+-----+ | par (barco, OT) asignado a v CONTRATISTA(*)
o GRUPO_ASTILLERO.RUBRO | ejecuta v REGISTRO_HORA (vincula
OPERARIO + BARCO + OT + | RUBRO + FECHA + HORAS + TIPO_TRABAJO)
v OPERARIO --> GRUPO_DE_TRABAJO --> RUBRO (principal)
```

10.2 Cardinalidad y reglas relacionales

- **La relación entre BARCO y OT es de muchos a muchos.** Muchas OTs son de catálogo, estandarizadas, que aplican a varios barcos (VARADERO para todo barco que sube a seco, SENTINA para todo barco que limpia su sentina). Esto permite estadística por tipo de trabajo. En consecuencia, la deducción de OT se filtra siempre por barco (RN-06).
- Una OT tiene asignados uno o más CONTRATISTAS o grupos ASTILLERO.RUBRO en la columna G de UNIFICADO, para un barco dado.
- Un OPERARIO pertenece a un único GRUPO_DE_TRABAJO; un grupo tiene múltiples operarios.
- Un OPERARIO tiene un RUBRO principal, pero puede ejecutar tareas de otros rubros (many-to-many en la práctica).
- Un REGISTRO_HORA vincula OPERARIO + BARCO + OT + RUBRO + FECHA + HORAS + TIPO_TRABAJO (columna F).

10.3 Notas para migración a modelo relacional

Al migrar a Supabase, entidades candidatas a tablas con claves foráneas: barcos, operarios, grupos_trabajo, rubros, ordenes_trabajo, ot_asignaciones (tabla puente entre barco y OT, dado que la relación es N:M), registros_horas (con tipo_trabajo explícito), log_procesamiento. La estructura de UNIFICADO, donde un par (barco, OT) es una fila, ya está en formato long y mapea bien a ot_asignaciones.

11. Manejo de errores

Filosofía: nunca silenciar un error de interpretación. Los errores se exteriorizan al usuario o quedan visibles en la planilla con el marcador R, pero no se ocultan.

11.1 Categorías de error y manejo

Categoría	Detección	Manejo
Fecha ausente	Regex sin match en E1.	Mensaje al emisor pidiendo fecha; flujo termina sin escribir.
Campo obligatorio vacío	needs_review = true en E4.	Fila escrita con R; motivo en columna AE.
OT sin habilitación	Ninguna candidata textual habilita al operario (RN-06).	Fila escrita con R; OT vacía; motivo en AE.
Mensaje del bot	body.event.bot_id presente en E1.	Se descarta con error controlado (evita loop).
Fragmento no asociado	El LLM lo reporta en campos_no_resueltos.	Fila adicional con R y texto literal en OBSERVACIONES.
Falla del LLM	Error de OpenAI.	Error capturado en E3c; fila de log marcada; mensaje al emisor.
Falla de Google Sheets	Error de la API (protección, grid limit).	Error visible; requiere intervención (ver Sección 18).

11.2 Política de degradación

El sistema favorece la degradación parcial sobre la falla total. Es preferible escribir 5 filas correctas y 2 con R que rechazar el mensaje completo. El encargado ya envió el mensaje; reenviarlo entero tiene un costo alto.

11 bis. Sistema de alertas (fase diferida)

Define el sistema de alertas dirigidas por rol. La estructura queda especificada y preparada en V1, pero su implementación se difiere: depende del catálogo de contactos por rol (Anexo D), que requiere la base de datos de Supabase. El nodo E9b se libera en V2.1, después de la migración.

En V1, todas las filas con R quedan visibles en la planilla y el resumen general se envía al encargado de registrar (E9). El sistema de alertas por rol agrega notificaciones segmentadas: cada responsable recibe únicamente lo que le compete.

11bis.1 Matriz de alertas por rol

Dato faltante	Destinatario	Rol	Bloqueante
OT no deducida	Jefe de obra de esa OT (columna H de UNIFICADO)	JEFE DE OBRA	No — fila con R
Contratista no asignado	Jefe de obra de esa OT	JEFE DE OBRA	No — fila con R
Barco no deducible	Encargado de registrar	ENCARGADO DE REGISTRAR	No — fila con R (RN-13b)
Fecha ausente	Encargado de registrar	ENCARGADO DE REGISTRAR	Sí — corta el mensaje (RN-11)
Operario no reconocido	Supervisor	SUPERVISOR	No — fila con R
Otros casos no resueltos	Supervisor	SUPERVISOR	No — fila con R

11bis.2 Lógica de agrupación y envío

Las alertas se agrupan y envían inmediatamente después de procesar cada mensaje (nodo paralelo a E9). La agrupación es por destinatario: si un jefe de obra tiene tres tareas con OT faltante, recibe un único mensaje consolidado, no tres. El envío es por usuario Slack individual (mensaje directo), no por canal compartido.

Formato de mensaje (ejemplo, jefe de obra)

Jefe [Nombre]: Faltan detalles para las siguientes tareas: - [Barco] - [Operario] - [Rubro]: falta OT - [Barco] - [Operario] - [Rubro]: falta asignación de contratista
Por favor detalla en el canal Slack [ID/nombre del canal].

11bis.3 Diseño de roles fijos, usuarios variables

El subsistema desacopla el rol del usuario. Los roles (JEFE DE OBRA, ENCARGADO DE REGISTRAR, SUPERVISOR) son fijos; las personas que los ocupan y sus identificadores Slack varían. El sistema resuelve en tiempo de ejecución qué usuario ocupa el rol relevante para cada alerta. Un cambio de personal solo requiere actualizar el catálogo de contactos (Anexo D), sin tocar la lógica del flujo.

12. Auditoría y trazabilidad

12.1 Principios de trazabilidad

- Mensaje literal preservado: el texto crudo se almacena íntegro en el log.
- Decisiones del agente registradas: cada fila lleva los campos `match_*` y el `motivo_revision` que explican cómo se resolvió cada entidad.
- Vinculación cruzada: el `mensaje_id` (columna AD) permite saltar desde una fila de la planilla al log y a todas sus filas hermanas.
- Trazabilidad de costos: `tokens_in` y `tokens_out` por mensaje.
- Versionado de prompt: `prompt_version` registra qué versión del agente generó qué resultado.

12.2 Cómo se gestionan las versiones del prompt

El prompt completo (rol, reglas, few-shot e instrucciones) vive en el nodo Set - PromptConfig, con dos campos editables desde la UI: `prompt_version` y `prompt_template`. Los catálogos se inyectan dinámicamente en E3a vía placeholders; el resto del prompt es fijo y versionado. Cada cambio incrementa `prompt_version` manualmente. Flujo recomendado para cambiar el prompt:

1. Antes de cambiar, correr el test de regresión (10-20 mensajes de referencia) con la versión actual.
2. Hacer el cambio e incrementar la versión (v1.1 → v1.2).
3. Correr los mismos tests con la nueva versión.
4. Si mejora, deployar. Si empeora, revertir al texto anterior.

12.3 Campo `mensaje_id` en planilla destino

Incluido en el alcance V1: la planilla registra el `mensaje_id` originador de cada fila en la columna AD. Permite, desde una fila individual, saltar al log y ver el mensaje completo junto con todas sus filas hermanas. Habilita la trazabilidad bidireccional entre planilla y log.

13. Dependencias críticas

13.1 Dependencias técnicas externas

Dependencia	Función	Riesgo si falla
Slack API	Recepción de mensajes (webhook) y respuesta al emisor.	Bloqueo total del trigger.
OpenAI API	Extracción semántica.	Bloqueo del procesamiento; sin alternativa inmediata.

Dependencia	Función	Riesgo si falla
Google Sheets API	Lectura de catálogos (B.D.NewSystemm, LISTA_TRABAJOS) y escritura (HORAS).	Bloqueo total de lectura y persistencia.
n8n self-hosted	Plataforma de ejecución (Docker, v2.4.7).	Caída del orquestador detiene el sistema.
Evolution API	(Futuro) WhatsApp; hoy no activa.	Sin impacto en V1.

13.2 Dependencias funcionales internas

- Calidad y vigencia del catálogo de personal: nuevos operarios deben ingresarse (y marcarse ACTIVO) antes de aparecer en mensajes.
- Calidad de la columna G de UNIFICADO: las asignaciones de contratistas deben mantenerse actualizadas para que RN-06 y RN-07 funcionen.
- Convención de nomenclatura estable en el catálogo de personal: si cambia el formato de nombres, el matching degrada.
- Disponibilidad de filas físicas en la hoja HORAS: la escritura por rango requiere que la hoja tenga filas vacías disponibles (ver Sección 18).

13.3 Punto único de falla más sensible

El catálogo de personal en B.D.NewSystemm es el punto único de falla más sensible. Si se corrompe, se renombra masivamente o se reorganiza, el sistema pierde su capacidad de matching. Como red de seguridad se propone un snapshot diario (no implementado en V1; descripción en la Sección 17.3).

14. Riesgos operativos

Riesgo	Prob.	Impacto	Mitigación
Drift del LLM (cambio de comportamiento de gpt-4o-mini).	Media	Alto	Versionado de prompt; pinning del modelo; tests de regresión.
Alucinación de operarios o barcos inexistentes.	Baja	Alto	Validación cruzada en E4; filas fuera de catálogo se marcan con R.
Costo de OpenAI fuera de presupuesto.	Baja	Medio	Tracking de tokens en log; alertas por umbral.
Loop de auto-respuesta del bot.	Resuelto	Alto	Filtro de body.event.bot_id en E1.
Filas duplicadas por reenvío del mismo mensaje.	Media	Bajo	Detección por messageTs; idempotencia opcional vía hash.
Hoja HORAS sin filas físicas disponibles.	Media	Medio	Extender filas manualmente; alerta de umbral propuesta (Sección 18).
Cambios de esquema en catálogos sin coordinación.	Alta	Alto	Documentar contrato CargarCatalogos (Anexo C); tests sobre el sub-workflow.

15. Escalabilidad futura

15.1 Ejes de escalabilidad

- **Volumen:** escalar a cientos de mensajes/día requiere caching de catálogos, procesamiento concurrente y, eventualmente, base de datos transaccional.
- **Canales de entrada:** el Webhook está diseñado para ser intercambiable. Agregar WhatsApp, email u otro canal solo requiere normalizar al objeto interno estándar.
- **Persistencia de catálogos:** CargarCatálogos absorbe la fuente. Migrar a Supabase implica reescribir solo ese sub-workflow.
- **Persistencia de registros:** E7a/E7b pueden migrar de Sheets a Supabase de forma similar.
- **Complejidad semántica:** si los catálogos superan ~3000 entradas, migrar a tool calling o RAG en lugar de catálogos en contexto.

15.2 Roadmap propuesto

Hito	Cambio	Trigger del cambio
V1.1	Correcciones post-MVP: OT por habilitación, escritura por rango, prompt editable, columna AE.	Implementado tras validación del MVP.
V1.2	Reglas de negocio por OT (rubros permitidos por tipo de orden).	Cuando se mapee la lógica de negocio del proyecto de aplicación.
V1.3	Mecanismo de pendientes para mensajes incompletos.	Si el % de mensajes rechazados por fecha es alto.
V2.0	Migración de catálogos a Supabase, incluida tabla de contactos por rol.	Cuando esté listo el modelo relacional.
V2.1	Activación de alertas por rol (E9b) y migración del trigger a WhatsApp.	Tras disponer de la tabla de contactos y validar Evolution API.
V2.2	Migración de planilla destino a base de datos + UI.	Cuando la planilla sea cuello de botella.

16. KPIs sugeridos

Indicadores calculables desde LOG_PROCESAMIENTO sin instrumentación adicional.

16.1 KPIs de procesamiento

KPI	Objetivo
Mensajes procesados por día	Monitorear volumen.
Filas escritas por mensaje (promedio)	Detectar variaciones inusuales.
Tasa de filas con R	< 15% saludable; > 30% requiere intervención (prompt, catálogos).
Tiempo de procesamiento promedio	< 10 segundos por mensaje.
Tasa de mensajes sin fecha	< 5% saludable.

16.2 KPIs de costo y calidad

- Costo de LLM por mensaje y costo total mensual (vía tokens del log).
- Tasa de mismatch en match_contratista (< 10%; valores altos indican catálogo desactualizado).
- Tasa de mensajes sin tareas extraíbles (< 2%).

16.3 Implementación recomendada de los KPIs

Para V1, los KPIs calculables desde el log se materializan en una hoja KPI_DASHBOARD en la misma planilla, con fórmulas (COUNTIF, AVERAGEIF, SUMIF) sobre LOG_PROCESAMIENTO. El objetivo de mediano plazo es persistir estos KPIs en una base relacional (Supabase) para evaluar el rendimiento por área. No se recomienda construir esa base desde el inicio: el volumen actual no lo justifica. La hoja es suficiente hasta superar ~10.000 registros en el log.

17. Recomendaciones arquitectónicas

17.1 Decisiones de diseño que conviene preservar

- **Separación entre extracción semántica y validación determinística:** el LLM interpreta lenguaje ambiguo; la aritmética, consolidación y validación contra catálogos viven en código. Base de la confiabilidad.
- **Sub-workflows como puntos de migración:** CargarCatalogos aísla la dependencia con Sheets. Mantener esta abstracción es clave para que la migración a Supabase sea de bajo riesgo.
- **Trazabilidad por defecto:** el log se diseñó como ciudadano de primera clase, permitiendo auditoría barata y A/B testing del prompt.
- **Validación bloqueante de fecha:** protege la integridad del registro horario.
- **Selección de OT por habilitación, no por texto:** evita escribir OTs incorrectas sin señalarlas. Es preferible una R explícita a un dato falso silencioso.

17.2 Recomendaciones a corto plazo

- Implementar un test de regresión semántica: 10-20 mensajes con su output esperado, ejecutado antes de cambiar el prompt.
- Mantener actualizado el contrato CargarCatalogos documentado en el Anexo C.
- Establecer una alerta automática si la tasa de filas con R supera 30% en 24 horas.
- Agregar una validación en E7a que alerte cuando queden menos de ~500 filas físicas libres en la hoja HORAS.

17.3 Recomendaciones a mediano plazo

- Migrar el catálogo de personal a Supabase: primer candidato natural por ser el más crítico y menos volátil.
- Implementar un dashboard (Looker Studio o similar) sobre LOG_PROCESAMIENTO y KPI_DASHBOARD.

- Diseñar el mecanismo de pendientes documentando casos de uso y de error.

Propuesta: snapshot diario del catálogo de personal (no implementado)

Propuesta para implementación futura, no incluida en V1. Sería un flujo n8n independiente, programado diariamente (ej. 23:59 hs), que: (1) lee la hoja de personal completa; (2) la escribe en una hoja de respaldo (SNAPSHOT_PERSONAL_AAAA-MM-DD o sobrescribiendo SNAPSHOT_PERSONAL_LAST); (3) opcionalmente compara con el día anterior y registra diferencias en DIFF_PERSONAL. Al migrar a Supabase, el versionado por fila reemplaza este mecanismo.

17.4 Antipatrones a evitar

- No agregar lógica de negocio dentro del prompt del LLM (cálculo de horas, decisiones de aprobación, reglas de OT). El LLM extrae; el código decide.
- No usar append en la planilla destino cuando hay fórmulas arrastradas: cuentan como datos y rompen el posicionamiento. Usar escritura por rango anclada a columna B.
- No depender de posiciones de columna por número en lógica futura; preferir headers cuando sea posible.
- No mezclar lógica de canal (Slack) con lógica de negocio. E1 normaliza; el resto del flujo no debe saber que existe Slack.
- No usar `$input.first()` en nodos Code que vengan después de un nodo insertado: referenciar el nodo de origen por nombre explícito (ver Sección 18).

18. Lecciones de implementación (post-MVP)

Esta sección documenta problemas concretos encontrados durante la puesta en producción del MVP y sus soluciones. Sirve como referencia para mantenimiento y para futuros flujos del astillero.

18.1 Trigger de Slack: credencial y handshake

El nodo Slack Trigger nativo requiere una credencial de tipo "Slack API" (Access Token + Signing Secret), distinta de la credencial "Slack OAuth2 API" usada por los nodos de acción. En la versión de n8n utilizada (2.4.7), el trigger no permitía seleccionar la credencial OAuth2 existente. La solución fue usar un nodo Webhook genérico con Event Subscriptions de Slack.

Slack valida la URL del webhook enviando un challenge que el endpoint debe responder. El patrón que funcionó: nodo Webhook con respuesta "Using Respond to Webhook Node", un nodo Respond to Webhook con Response Body en modo Text conteniendo la expresión que lee el challenge desde el nodo Webhook por referencia explícita. Una vez verificada la URL, Slack no vuelve a enviar el challenge, por lo que ese mecanismo solo es necesario para la verificación inicial.

El bot debe ser invitado explícitamente al canal con `/invite @nombrebot`. Sin esa invitación, el webhook se verifica pero no llegan mensajes.

18.2 Loop de auto-respuesta del bot

Tras conectar el trigger, el bot entró en loop: su propia respuesta "falta fecha" era publicada en el canal, lo que disparaba una nueva ejecución que volvía a responder. El payload de esos mensajes incluye

body.event.bot_id. La solución fue filtrar en E1: si body.event.bot_id está presente (o subtype = bot_message), el mensaje se descarta.

18.3 Estructura del payload: body.event

El texto del mensaje no está en la raíz del payload sino anidado en body.event.text. El código de E1 debe leer desde body.event, no desde la raíz. Lo mismo aplica a user, ts y channel.

18.4 Referencias entre nodos Code: \$input vs nodo por nombre

Lección recurrente: en n8n, un nodo Code con \$input.first().json toma el JSON del nodo inmediatamente anterior. Cada vez que se inserta un nodo nuevo entre dos existentes, cualquier Code posterior que use \$input se rompe silenciosamente, porque su origen cambió. La forma robusta es referenciar el nodo de datos por nombre explícito, por ejemplo \$('E2 - CargarCatalogos').first().json. Este error apareció dos veces durante la implementación (E3a y E7a) tras insertar los nodos Set - PromptConfig y GET - LeerColumnaB.

18.5 Escritura en planilla: protección, append y grid limits

- **Celdas protegidas:** la hoja tenía rangos protegidos que bloqueaban la escritura. Se resolvió agregando la cuenta de servicio a los permisos del rango en Datos → Hojas y rangos protegidos.
- **append salta filas:** el método append cuenta las fórmulas arrastradas (columna S) como datos y escribe después de ellas, no después de la última fila real. Se reemplazó por escritura por rango (update PUT) anclada a la última fila con valor en columna B.
- **grid limits:** la escritura por rango falla si la hoja no tiene filas físicas disponibles (error "exceeds grid limits"). Hay que extender la hoja con filas vacías. Recomendación: mantener siempre un margen de filas libres.
- **matchingColumns en append:** la operación appendOrUpdate de Google Sheets requiere el parámetro matchingColumns. Para el log, que solo agrega, conviene usar la operación append simple.

18.6 Colapso de items en sub-workflows

En CargarCatalogos, los nodos Code - Collapse1 y Collapse2 existen para colapsar los N items de un catálogo en 1 solo, evitando que el siguiente nodo de lectura se ejecute una vez por cada fila. No transforman datos: solo controlan la cardinalidad del flujo. Los datos reales se recuperan después con \$('GS - Leer ...').all() en el nodo BuildCatalog.

Anexo A — Glosario operativo

Término	Definición
Astillero	Instalación de construcción, reparación y mantenimiento de embarcaciones.
Barco	Embarcación que ingresa al astillero. Unidad de agrupación primaria de los trabajos.
Orden de Trabajo (OT)	Definición de una tarea. Muchas OTs son de catálogo (VARADERO, SENTINA) y aplican a varios barcos; la relación barco-OT es muchos a muchos.
Rubro	Categoría de trabajo (raschinaje, calderería, carpintería, mecánica, pintura, pala, grúa, etc.).
Operario propio	Empleado del astillero. Grupo de trabajo = ASTILLERO. Target 9h L-V.
Contratista externo	Persona o empresa subcontratada. Grupo distinto a ASTILLERO. Target 8h L-V.
Encargado de registrar	Persona que recibe los reportes por WhatsApp y los publica en Slack. Emisor registrado en el log.
Marcador R	Identificador en columna APROBACIÓN que señala que la fila requiere revisión humana.
Habilitación	Que un operario/grupo figure en la columna G de UNIFICADO para una OT, lo que lo autoriza a trabajarla.
match_*	Familia de campos de auditoría que registran cómo se resolvió cada entidad (exact, fuzzy, deduced, none).
motivo_revision	Diagnóstico del sistema sobre por qué una fila quedó con R (columna AE).
raw_block	Fragmento literal del mensaje original que dio lugar a una fila específica.
Contrato CargarCatalogos	Forma estable de los catálogos en JSON, independiente de su fuente real.

Anexo B — Ejemplos de mensajes y output esperado

B.1 Ejemplo: mensaje de operador de pala

Mensaje recibido:

22-5 Pala sacar tobera de cáliz y llevar al fondo 30 minutos tobares. Pala tirar chapa en desguace Ávalos 1 hora Pala llevar oxígeno a don Vicente 30 minutos nieri.

Interpretación esperada:

- Fecha: 22/05/{año}. Rubro común: PALA. Operario ejecutor: Hernández, Santiago (maquinista, deducido por rubro PALA).
- Línea 1: barco CALIZ, OT deducida (TOBERA), 0.5h.
- Línea 2: barco AVALOS, OT deducida, 1h.
- Línea 3: barco DON VICENTE, OT deducida, 0.5h.

B.2 Ejemplo: reporte multi-operario por equipo

Mensaje recibido (fragmento):

RaschinesFranco e higinio Achique y limpieza centina Caliz 9 hs Luciano enzo Achique pozo. Pezca Lobo 1 hs

Interpretación esperada:

- Rubro contextual: RASCHINAJE.
- Bloque 1: dos operarios (Rancatti, Franco y Sostoa, Higinio), barco CALIZ, OT achique/limpieza, 9h cada uno. OBSERVACIONES corregido a "Achique y limpieza sentina".
- Bloque 2: dos operarios (Gonzalez, Luciano y un Enzo), barco LOBO, OT achique pozo, 1h cada uno.
- Cualquier ambigüedad de homónimos se resuelve por rubro contextual (RN-04 punto 3); si no resuelve, queda con R.

B.3 Ejemplo: corrección de OT por habilitación (caso real)

Mensaje recibido (fragmento): "16/6 Gary Andres, vuoso parche sentina 9hr".

Gari, Mario Andres es del grupo ASTILLERO con rubro CALDERERIA. En el barco DON VICENTE VUOSO existen varias OTs cuya descripción menciona "sentina":

- OT SENTINA — habilita solo ASTILLERO.RASCHINAJE.
- OT CALDERERIA VARIOS — habilita ASTILLERO.CALDERERIA, y su descripción incluye "Parche en sentina".

Interpretación esperada: el sistema descarta SENTINA (no habilita calderería) y resuelve CALDERERIA VARIOS, la única que habilita al operario. La versión inicial escribía SENTINA por coincidencia textual; RN-06 corrige esto.

Anexo C — Contrato de datos CargarCatalogos

Especifica las fuentes, columnas y filtros que alimentan el sub-workflow CargarCatalogos en la implementación sobre Google Sheets. El contrato de salida (forma del JSON) se mantiene estable; lo que cambiará al migrar a Supabase es la capa de lectura.

C.1 Spreadsheets involucradas

Spreadsheet	ID	Rol
B.D.NewSystemmm	1OHFcKWKOWaknei6eV6wBZrEXHlye2QK3RJUqC0yDT-4	Fuente de personal y rubros.
LISTA_TRABAJO	1rmjCqJoy7Iz0YPlbmzHxp9Jj_pMHfSNBmpvS3BhxAlQ	Fuente de barcos y OTs activas (hoja UNIFICADO).
HORAS_PLANILLAS_DE_REGISTRO	19SFRudDicJ2gKPCWW-UDu2J7-JlpZlrW46TX1DBgmuw	Solo escritura: planilla destino y log. No se lee como catálogo.

C.2 Catálogo: Personal

Fuente: B.D.NewSystemmm → hoja PersonalClasificacionCostos.

Filtro: solo registros con ESTADO (columna H) = ACTIVO.

Columna	Header	Campo JSON
A	NOMBRE	nombre
B	GRUPO DE TRABAJO	grupo
C	U\$/HORA	usd_hora
D	RUBRO	rubro
H	ESTADO	(filtro)

C.3 Catálogo: Rubros

Fuente: B.D.NewSystemmm → hoja BaseDatosP/PlanillasRegistro, columna C.

Es la fuente de verdad de los rubros. La hoja RUBROS_OtrosDesplegables_IMPORTADA es un export derivado de esta columna; por eso no se usa como fuente de lectura.

C.4 Catálogo: Barcos

Fuente: LISTA_TRABAJO → UNIFICADO, columna A.

Filtro: valores únicos no vacíos, considerando solo filas con estado (columna E) = "En ejecución/En curso".

C.5 Catálogo: OTs activas

Fuente: LISTA_TRABAJOS → UNIFICADO.

Filtro: estado (columna E) = "En ejecución/En curso". Distinto del filtro de AvisoOtNueva (columna L), que son dos vistas distintas de la misma hoja.

Columna	Header	Campo JSON
A	BARCO	barco
C	ORDEN DE TRABAJO	ot
D	Descripcion	descripcion
G	Contratista / Externo	contratistas_col_g
F	Comentario	(infiere tipo_trab, RN-14)
H	JEFE DE OBRA	jefe_obra
E	Estado	(filtro)

Tipo de trabajo: se infiere del texto de la columna Comentario (F) según RN-14. Vacío → HORA; mención de presupuesto → PRESUPUESTO; mención de control → CONTROL. No es una columna numérica dedicada sino una lectura semántica del comentario.

C.6 Referencia de headers de UNIFICADO

Col	Header	Col	Header
A	BARCO	K	usuario_origen
B	FECHA	L	enviar notificacion
C	ORDEN DE TRABAJO	M	ENVIO AVISO
D	Descripcion	T	SlackChannel_ID
E	Estado	U	SeCreoOT
F	Comentario		
G	Contratista / Externo		
H	JEFE DE OBRA		
J	NUMERO DE OT		

Anexo D — Catálogo de contactos por rol (preparado para V2.1)

Estructura propuesta para el catálogo que el sistema de alertas (Sección 11 bis, nodo E9b) consultará. No se implementa en V1: la estructura definitiva vivirá en Supabase y se construirá en V2.1.

Estado: propuesta. La estructura está sujeta a revisión. Lo firme es el principio de diseño (rol fijo, usuario variable) y los campos mínimos. La decisión sobre dónde alojar provisionalmente la lista queda abierta.

D.1 Principio de diseño

El catálogo desacopla el rol del usuario que lo ocupa. La lógica razona en términos de roles (JEFE DE OBRA, ENCARGADO DE REGISTRAR, SUPERVISOR), estables; el catálogo traduce el rol al usuario Slack concreto en tiempo de ejecución. Un cambio de personal solo actualiza este catálogo.

D.2 Estructura propuesta de la tabla

Campo	Tipo	Descripción
rol	enum	JEFE DE OBRA ENCARGADO DE REGISTRAR SUPERVISOR.
clave_vinculo	string	Valor que vincula el rol a una entidad. Para JEFE DE OBRA, el nombre de la columna H de UNIFICADO.
nombre_persona	string	Nombre legible de la persona que ocupa el rol.
usuario_slack_id	string	ID de usuario Slack para mensaje directo.
canal_slack_id	string	Canal Slack donde se solicita el detalle (opcional).
grupo_trabajo	string	Grupo de trabajo del usuario.
activo	boolean	Permite desactivar un contacto sin borrarlo.

D.3 Resolución en tiempo de ejecución

Para una fila con R por OT no deducida o contratista no asignado, el sistema toma el JEFE DE OBRA de la columna H de UNIFICADO para esa OT, lo usa como clave_vinculo y obtiene el usuario_slack_id. Para barco faltante, resuelve ENCARGADO DE REGISTRAR. Para operario no reconocido u otros, SUPERVISOR. Si la clave no encuentra contacto, la alerta cae a un destinatario por defecto (a definir) y se registra la ausencia.

D.4 Pendientes de esta etapa

- Definir la fuente y estructura definitiva de la tabla en Supabase.
- Cargar el mapeo rol → usuario_slack_id para cada jefe de obra, encargado y supervisor.
- Definir la lista de canales Slack por rol.
- Definir el destinatario por defecto para casos sin contacto resuelto.

Anexo E — Mapa de nodos del flujo n8n implementado

Lista de nodos del workflow principal RegistroHoras_Aloncar_V1 tal como quedó implementado, con su tipo y función. El sub-workflow CargarCatalogos se detalla al final.

E.1 Workflow principal

Nodo	Tipo n8n	Función
Webhook	webhook	Recibe eventos de Slack. Responde el challenge inicial.
E1 - ExtractAndValidate	code	Lee body.event, filtra bot, extrae fecha, genera mensaje_id.
IF - Tiene Fecha	if	Bifurca según haya fecha o no.
E1b - Solicitar Fecha	slack	Pide fecha al emisor si falta. Termina el flujo.
E2 - CargarCatalogos	executeWorkflow	Invoca el sub-workflow de catálogos.
Set - PromptConfig	set	prompt_version + prompt_template editables.
E3a - PrepararPrompt	code	Inyecta catálogos en el template del prompt.
E3b - OpenAI	httpRequest	Llama a la API de OpenAI (gpt-4o-mini).
E3c - ParseLLMResponse	code	Parsea el JSON del LLM, fusiona con datos del mensaje.
E4 - ValidacionCruzada	code	Matching, OT por habilitación, tipo_trab, needs_review, motivo_revision.
E5 - CalcularHoras	code	Distribución normal/50/100 según calendario y tipo.
E6 - ConsolidarFilas	code	Agrupar por entidad; separa observación y motivo_revision.
E6b - PrepararLecturaColB	code	Arma URL para leer columna B de HORAS.
GET - LeerColumnaB	httpRequest	Lee la columna B para detectar última fila real.
E7a - PrepararSheet	code	Calcula rango de escritura y matriz A:AE.
E7b - UpdatePlanilla	httpRequest	Escribe por rango (PUT update).
E8 - EscribirLog	googleSheets	Append de una fila en LOG_PROCESAMIENTO.
E9a - PrepararRespuesta	code	Arma el texto de confirmación para Slack.
E9 - ResponderUsuario	slack	Envía la confirmación al emisor.
E9b - Alertas PLACEHOLDER	noOp	Diferido a V2.1 (Sección 11 bis).

E.2 Sub-workflow CargarCatalogos

Nodo	Tipo n8n	Función
Trigger - ExecuteWorkflow	executeWorkflowTrigger	Punto de entrada del sub-workflow.
GS - Leer Personal	googleSheets	Lee PersonalClasificacionCostos.
Code - Collapse1	code	Colapsa N items en 1 (control de cardinalidad).
GS - Leer Rubros	googleSheets	Lee BaseDatosP/PlanillasRegistro.

Nodo	Tipo n8n	Función
Code - Collapse2	code	Colapsa N items en 1.
GS - Leer UNIFICADO	googleSheets	Lee UNIFICADO (barcos + OTs).
Code - BuildCatalog	code	Construye el objeto JSON de catálogos con todos los filtros.

Credenciales requeridas al importar: Slack API (trigger/respuestas), Google Sheets OAuth2 (lecturas y escrituras), OpenAI (E3b). El ID del sub-workflow debe cargarse en el nodo E2 tras importar CargarCatalogos. El path del webhook debe coincidir con el registrado en Slack Event Subscriptions.

Metadata del documento

Generado: Junio 2026.

Versión: 2.0 (final) — MVP implementado y validado en producción. Incorpora el diseño completo, las correcciones post-MVP (selección de OT por habilitación, escritura por rango, prompt editable, columna AE) y las lecciones de implementación.

Audiencia: arquitectos de sistemas, ingenieros de procesos, supervisores funcionales, equipos de mantenimiento de la automatización.

Próximos hitos: reglas de negocio por OT (V1.2), migración a Supabase (V2.0), sistema de alertas por rol y trigger WhatsApp (V2.1).